

Chapter #10

Data-Staging Design

→ Data staging design is about planning how to load data from operational systems into data marts.

→ if the reconciled layer is available, the population process is divided into two phases:

- from the operational sources to the reconciled database.

- From the reconciled database to the data mart.

Populating Reconciled Databases:

→ This involves fill the reconciled database with data from different sources.

Reconciled databases:

↳ it is like staging area where data from different sources is cleansed, transformed and prepared before it's moved to the final destination.

How does it work?

Staging area:

⇒ A temporary area where "raw" data stored before being cleansed and transformed.

⇒ it can take up a lot of space. So, it's important to plan for its size.

⇒ Three tasks should be associated with the data cleansing procedures, which help eliminate the inconsistencies due to errors and failures.

Extraction:

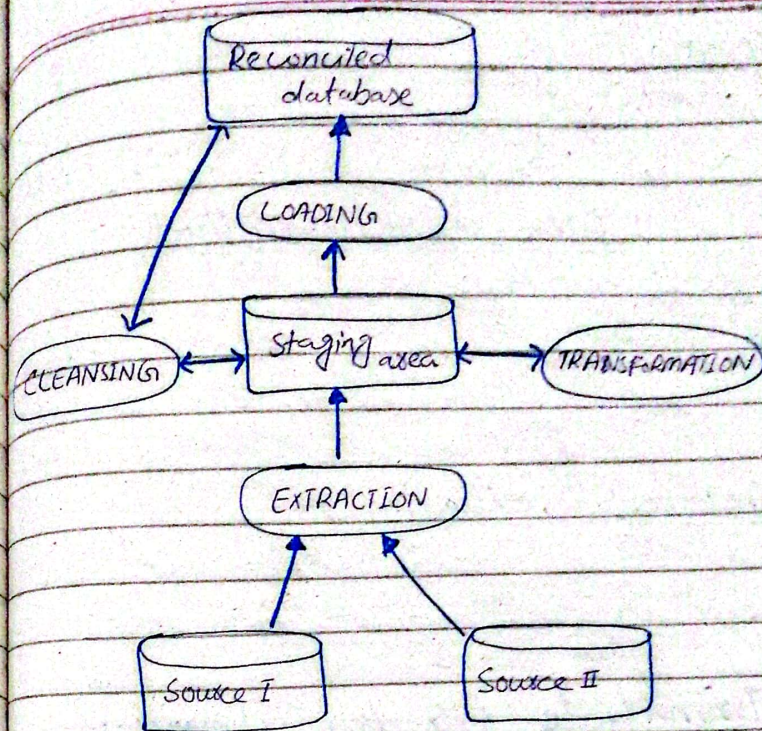
○ Tasks aimed at collecting data from sources.

Transformation:

○ Tasks aimed at adjusting the format of data from source schemata to reconciled schemata.

Loading:

○ Tasks aimed at entering the transformed data into the reconciled database and updating the already existing data.



Data extraction:

→ Data is collected from different sources.

→ There are three different types of data:

Transient data:

○ Only the current snapshot is kept.

Semi-Periodic data:

○ A limited number of past versions are kept.

Temporal data:

- o All changes are tracked for a defined period.

Data extraction methods

1. Static extraction:

- o Scans all data in the source database. It's good for initial setup but can be slow for large datasets.

2. Incremental Extractions:

- o Only extracts changed data. It is more efficient for frequent updates.

Immediate Extraction Techniques:

⇒ These techniques record changes to data as soon as they happen in the operational database.

1. Application-Assisted Extraction:

- o Creates a set of functions in the operational applications to store data changes to the staging area.
- o It can be used for legacy systems that don't support triggers, logs, & timestamps.

Pros:

o can capture changes in real-time and highly flexible.

2. Trigger-based extractions:

- o This technique uses triggers in the database to automatically capture changes to data.

o A trigger is like a rule that is activated when a specific event occurs, such as when a record is inserted, updated or deleted.

o When a trigger is activated, it stores the modified data in a special file or table, which can then be retrieved later for processing.

3. Log-based extraction:

- o This technique uses the database log files to capture changes to data.
- o Log files are created by the database to record all changes made to data.

o By analyzing the log files, identify changes that have been made & extract the relevant data.

4. Timestamp-based extraction:

⇒ This technique relies on timestamps to identify which data has been modified since the last extraction.

How it works?

Timestamping:

- Each record in the database has a timestamp associated with it, indicating when it was last updated.

Extraction:

- The extraction process compares the timestamps of records to the last extraction time.

Data selection:

- Only records with timestamps newer than the last extraction time are extracted.

5. FILE Comparisons:

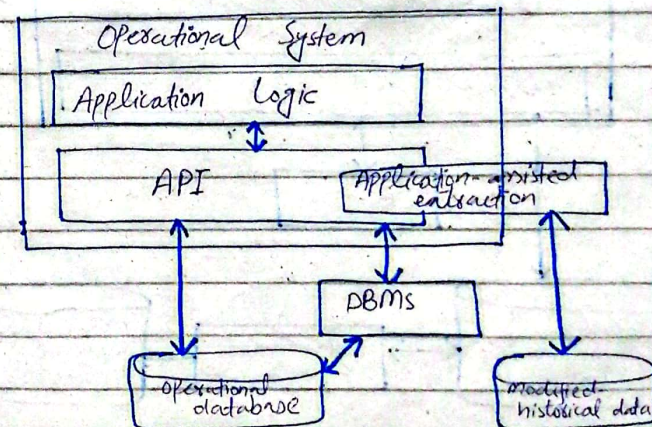
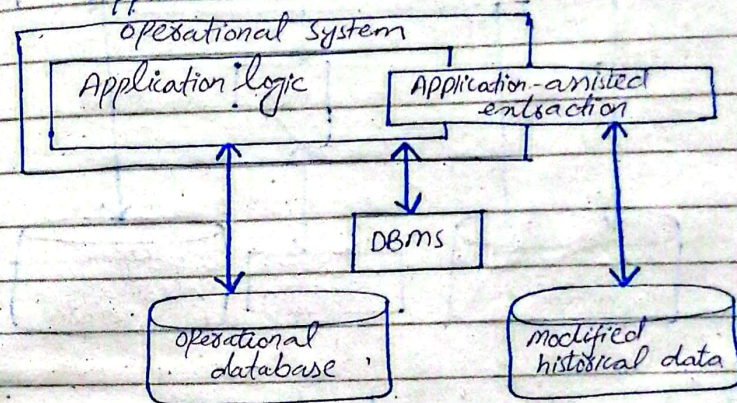
- Compare file versions to identify changes.
- This is delayed extraction technique.
- This requires that, after each extraction, the extracted data be kept in the staging area until the next extraction.

to enable its comparison to the new data extracted.

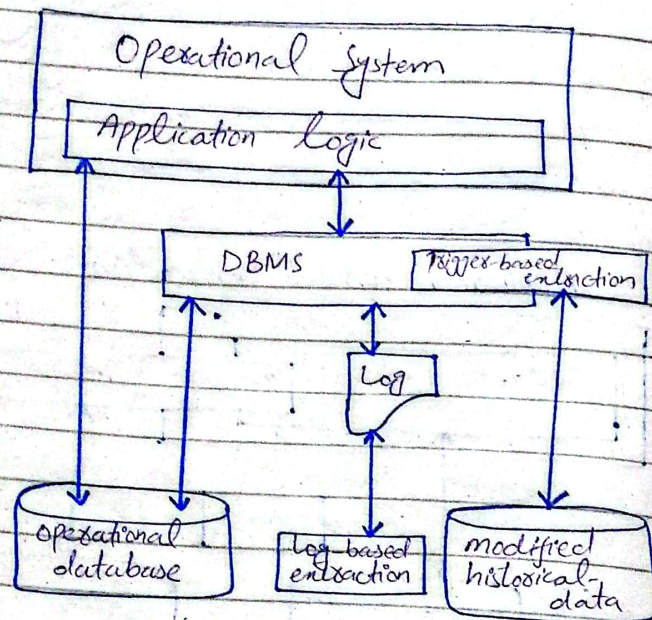
- The technique is normally used when operational data storage is file based or when DBMS does not support triggers or logs.

Diagrams:

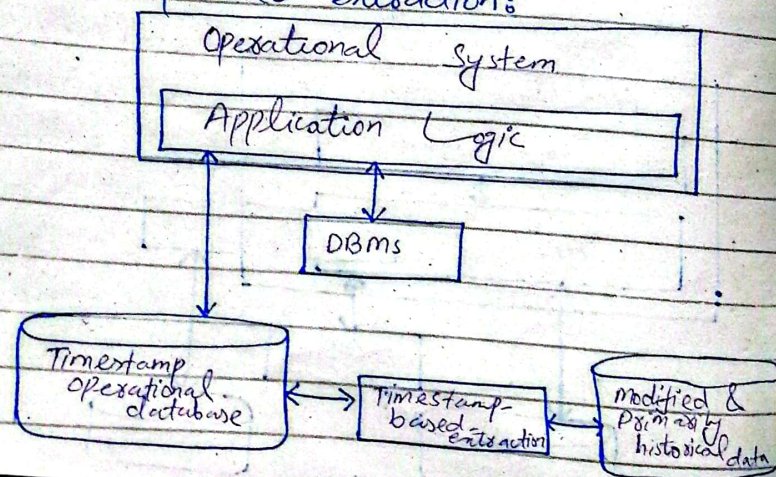
Application-assisted extraction



Extraction based on triggers and time-stamps



Timestamp based extraction:



Transforming Data:

Data is transformed to fit the target data warehouse or data mart schema.

Common types of transformation are as follows

1. Conversions

This involves changing the data type, format or measurement unit of an attribute.

ex:

- Converting dates from one format to another, converting currency from one format to another.
- Write name of a company in different ways.
- 'Coca-Cola', 'Coca-Cola Co' & so on.

2. Enrichment:

This involves adding new information to the data by combining existing attributes or by calculating new ones.

example:

- Calculating a customer's total purchase amount.

Separation/Concatination:

- o This transformation involves separating data that is stored together in the source system but needs to be kept separate in the reconciled database.
- o This often done through normalization & denormalization process.

Loading data

- ⇒ This is the final step in the data staging design process.
- ⇒ Data that has been extracted and transformed is loaded from the staging area into the reconciled database.
- ⇒ This type of loading process depends on the extraction technique used and the nature of the data being loaded (temporal or non-temporal).

Cleansing data & Various Techniques

⇒ Data cleansing, also known as data cleaning or data scrubbing, is the process of detecting and correcting errors and inconsistencies in a dataset. It ensures that data is accurate, consistent and ready for analysis.

Why important?

- o Data quality
- o Data integrity
- o Data consistency

Common data cleansing Tasks:

- o Identifying and correcting errors
- o Standardizing data
- o Removing duplicates
- o Validating data
- o Enriching data

Notes:

- o While data cleansing can be seen as a part of data transformation, it's often treated separately due to its significant impact on data quality & the complexity of the tasks involved.

Common causes of data inconsistency

⇒ Some common causes are:

1. Typing Mistakes:

- Simple errors during data entry like typos.
- Implementing data validation checks can help prevent these errors.

2. Inconsistency b/w attribute value & description:

- Discrepancies between related data elements occur when data is entered into the wrong attribute due to changes in data requirements.

- For example, entering a mobile number in a landline phone number field.

3. Inconsistency b/w correlated attributes:

- When values in related attributes are inconsistent.
- For instance, a person's city being "Miami" and their state being "California".
- Data validation rules & cross-checking can help identify and correct such inconsistencies.

3. Missing information:

Users may leave fields blank due to lack of time or understanding of their importance.

Mitigating data inconsistency

- Enforce strict data entry rules
- Data validation checks
- Data cleansing
- Data quality management.

Data Cleansing Techniques

⇒ There are various techniques for data cleansing:

1. Dictionary-based Techniques:

⇒ Compares data values to a predefined dictionary or lookup table to check for validity.

Use cases:

- Identifying & correcting typos.
- Standardizing data formats
- Ensuring consistency b/w related attributes.

Example: Checking if a city name is valid by comparing it to a list of valid cities.

2. Approximate Merging:

⇒ Matches records from different sources that might have slight variations in data.

Use cases:

- Merging data from different systems with overlapping data.
- Identifying duplicate records with minor variations.

Example:

- Matching customer records from two databases based on similar names & addresses, even if the exact information might not match perfectly.

3. Ad-hoc Techniques:

⇒ Ad-hoc techniques are used when specific business rules or domain-specific checks are required that cannot be handled by standard tools.

⇒ These techniques involve writing custom queries or scripts to ensure data integrity and consistency.

Ex:

○ Financial data:

Verifying that the equation $\text{Profit} = \text{revenue} - \text{expenses}$ holds true for all records.

$\text{Capital} = \text{assets} + \text{credits} - \text{debits}$

- functional dependency check
- outliers detection

Data Cleansing Tools:

○ Open source tools:

→ Python (Pandas, NumPy), R

○ Commercial tools:

- Tableau Prep, Alteryx, Trifacta.

(Continue this chapter on Register 3)

Populating Dimension Tables

⇒ Dimension tables are the first tables to be populated in a data warehouse.

⇒ This is because they form the basis of the star schema structure, where fact tables reference dimension tables to provide context.

Key steps to populating:

1. Identifying data to load

- Determine which attributes from the reconciled database are relevant to the dimension table.

2. Replacing keys

- Assign surrogate key to new tuples in the dimension table.
- Keep a mapping between the surrogate key and the primary key of the reconciled database tuple. This helps track changes & updates.

Handling different Hierarchy

Types:

Static hierarchies:

- o Only new tuples can be added, existing tuples cannot be modified.

Type 1 Hierarchies:

- o Overwrite existing tuples with new values, effectively losing historical data.

Type 2 Hierarchies:

- o Creating new tuples for each change, preserving historical data.

Type 3 Hierarchies:

- o Store a limited number of versions of each attribute.

Fully-logged Hierarchies:

- o Log all changes to dimension table tuples, including historical data.

Incremental vs Static Population:

Incremental:

- o Updates only the changed data which is more efficient for large datasets.

Static:

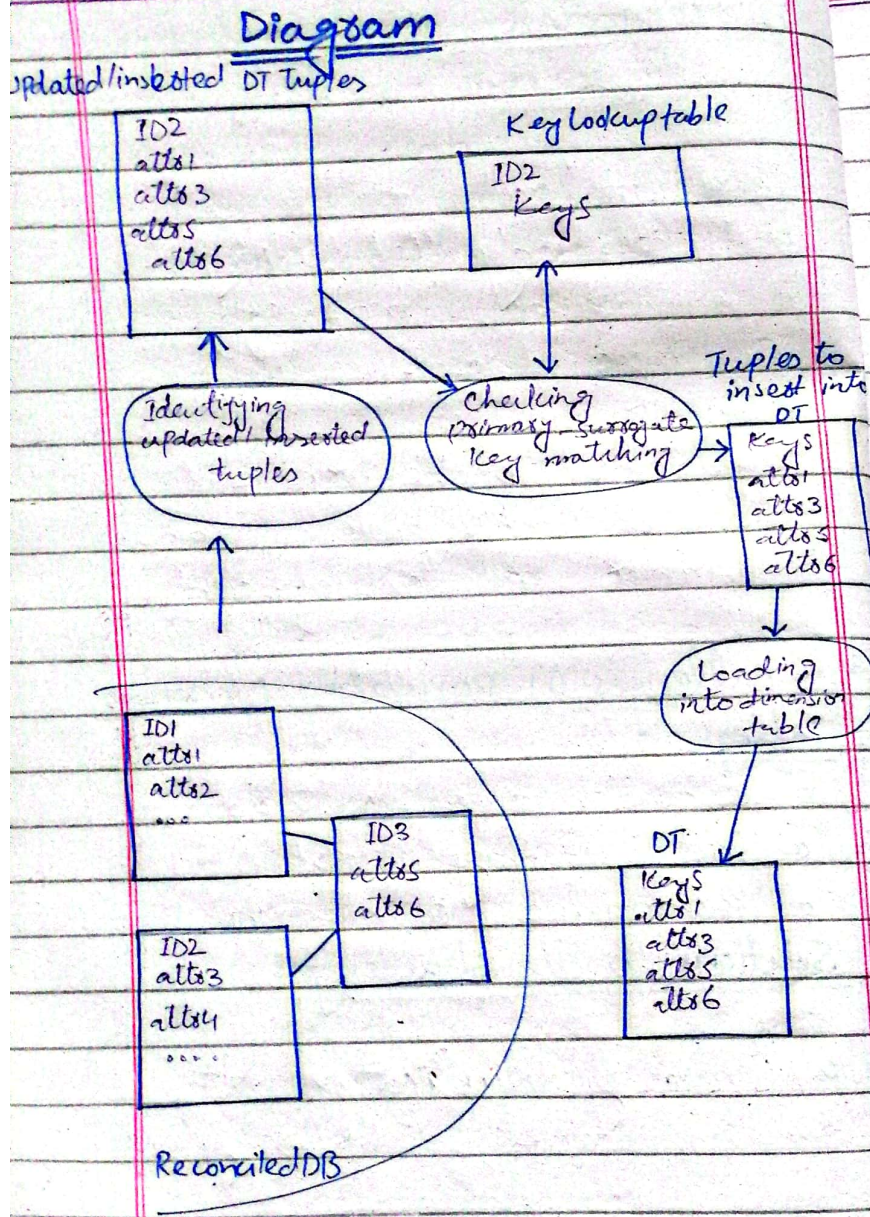
- o Repopulates the entire dimension table, which can be simpler but less efficient.

Challenges in populating Dimension Tables:

- o Complex data structures
- o Data quality issues
- o Performance optimization

Solutions to the challenges:

- o Temporary duplication of dimension table in the staging area.
- o Comparing to source data.



Populating Fact tables

⇒ Populating fact tables is similar to populating dimension tables. But there are some differences.

Order of operations:

- Should always populate dimension tables before populating fact tables. This ensures that the foreign keys in the fact tables reference valid entries in the dimension tables.

Key lookup tables:

- Use key lookup tables to map primary key from the reconciled database to surrogate keys in the dimension tables. This helps maintain data consistency and integrity.

Fact table measures:

- Determine the measures that will be stored in the fact

table. These measures should be derived from attributes in the reconciled schema.

Data changes:

- Fact tables can be updated to reflect changes in the reconciled data.

Change tracking:

- Use flags or timestamps to track changes to fact table tuples.

Efficiency:

- Use incremental updates to avoid unnecessary processing & improve performance.

Data Security:

- Protect sensitive data by implementing appropriate security measures.

Data quality:

- Ensure quality of data in fact table.

Semantic Considerations:

- Consider the semantics of different types of changes.
- This will help to determine how to handle updates and maintain data consistency.

Note: Diagram is same like populating dimension table.

Except it has more than one lookup tables.

Populating Materialized Views:

Materialized views are pre-computed results of SQL queries that are stored in the database.

They can significantly improve query performance, especially for complex &

frequently executed queries.

How they populated?

- Initial Population:
- o The materialized view is created by executing the underlying SQL query.
 - o The results of the query are stored in the materialized view.

Incremental updates:

- o To keep the materialized view up-to-date, incremental updates are applied.

Challenges:

- o Complexity
- o Performance ↓ due to frequent updates
- o Data Consistency

Strategies for efficient population:

- o Optimized query execution
- o Incremental updates
- o Parallel Processing
- o Caching

SALES	KeyS	KeyD	KeyP	quantity	receipts
	1	1	1	170	85
	2	2	2	320	160
New tuple →	3	1	3	412	200

STORE	Keys	store	City	state
	1	COOP1	Columbus	ohio
	2	COOP2	Austin	Texas
	3	COOP3	Dayton	ohio

PRODUCT	KeyP	Product	Type	category	brand
	1	Slush Milk	Dairy Product	Food	Slush
	2	Choko	Sweets	Food	mittels
	3	Yum Yum	Dairy Product	Food	Yum

DATE	KeyD	date	month	year
	1	2/9/2009	9/2009	2009
	2	3/10/2009	10/2009	2009

SALES	KeyS	KeyD	KeyP	quantity	receipts
modified tuple →	1	1	1	582	285
	2	2	2	320	160